

Instructions

6 exercises — prototype chain se lekar mixin patterns tak.

Code padhein, output predict karein, console mein verify karein.

Starred (*) exercises interview mein often pooche jaate hain — carefully karo.

Exercise 1 ★ — Prototype Chain Trace (Interview Favorite)

Concept: [Prototype Chain](#) **Task:** Har console.log ka output predict karo BEFORE running — mentally trace the chain:

```
function Animal(name) { this.name = name; }
Animal.prototype.speak = function() { return `${this.name} speaks`; };

function Dog(name, breed) {
  Animal.call(this, name); // borrow constructor
  this.breed = breed;
}
Dog.prototype = Object.create(Animal.prototype);
Dog.prototype.constructor = Dog; // fix constructor ref
Dog.prototype.bark = function() { return `${this.name}: Woof!`; };

const d = new Dog('Rex', 'Lab');
console.log(d.speak()); // Q1: ?
console.log(d.bark()); // Q2: ?
console.log(d instanceof Dog); // Q3: ?
console.log(d instanceof Animal); // Q4: ?
console.log(d.hasOwnProperty('name')); // Q5: ?
console.log(d.hasOwnProperty('speak')); // Q6: ?
console.log(d.constructor === Dog); // Q7: ?
```

Key Insight / Answer

Q1: 'Rex speaks' — inherited from Animal.prototype

Q2: 'Rex: Woof!' — own from Dog.prototype

Q3: true — d is instance of Dog

Q4: true — Dog.prototype chain includes Animal.prototype

Q5: true — 'name' was set on instance via Animal.call(this,name)

Q6: false — 'speak' is on Animal.prototype, not own property

Q7: true — we fixed constructor ref on line 8

Exercise 2 — Class vs Prototype Output

Concept: ES6 Class Internals **Task:** Class sirf syntactic sugar hai. Prove karo ki class aur constructor function same result dete hain:

```
// ES6 Class
class Circle {
  constructor(radius) { this.radius = radius; }
  area() { return Math.PI * this.radius ** 2; }
  perimeter() { return 2 * Math.PI * this.radius; }
  static unitCircle() { return new Circle(1); }
}

const c = new Circle(5);
// Verify class IS prototype under the hood:
console.log(typeof Circle); // Q1: ?
console.log(typeof Circle.prototype.area); // Q2: ?
console.log(c.hasOwnProperty('area')); // Q3: ?
console.log(c.hasOwnProperty('radius')); // Q4: ?
console.log(Object.getPrototypeOf(c) === Circle.prototype); // Q5: ?
// Method on prototype – shared, not copied!
const c2 = new Circle(3);
console.log(c.area === c2.area); // Q6: ? (same ref or different?)
const unit = Circle.unitCircle();
console.log(unit.radius); // Q7: ?
```

Key Insight / Answer

Q1: 'function' — class IS a function under the hood!

Q2: 'function' — methods are functions on prototype

Q3: false — area is on Circle.prototype, NOT instance

Q4: true — radius set in constructor → own property

Q5: true — confirms class uses prototype chain

Q6: true — SAME function reference! Not copied per instance — memory efficient

Q7: 1 — static factory method returns new Circle(1)

Exercise 3 ★ — Build a Shape Hierarchy

Concept: [extends & super](#) **Task:** Shape → Rectangle → Square hierarchy banao. Neeche diya structure follow karo:

```
class Shape {
  constructor(color = 'black') {
    this.color = color;
    this.id = Math.random().toString(36).slice(2, 7);
  }
  area() { return 0; }
  perimeter() { return 0; }
  describe() {
    return `${this.constructor.name} [${this.id}] | ` +
    `Color: ${this.color} | Area: ${this.area().toFixed(2)}`;
  }
}

class Rectangle extends Shape {
  constructor(width, height, color) {
    super(color); // Shape's constructor
    this.width = width;
    this.height = height;
  }
  area() { return this.width * this.height; }
  perimeter() { return 2 * (this.width + this.height); }
  isSquare() { return this.width === this.height; }
}

class Square extends Rectangle {
  constructor(side, color) {
    super(side, side, color); // Rectangle(side, side)
  }
  // All methods inherited – area, perimeter, describe
  resize(newSide) {
    this.width = this.height = newSide;
    return this; // method chaining
  }
}

const rect = new Rectangle(10, 5, 'blue');
const sq = new Square(7, 'red');
rect.describe(); // 'Rectangle [...] | Color: blue | Area: 50.00'
rect.perimeter(); // 30
rect.isSquare(); // false
sq.describe(); // 'Square [...] | Color: red | Area: 49.00'
sq.isSquare(); // true (inherited from Rectangle)
sq.resize(10).area(); // 100 – method chaining
sq instanceof Square; // true
sq instanceof Rectangle; // true
sq instanceof Shape; // true
```

Key Insight / Answer

Square extends Rectangle → Rectangle extends Shape (3 levels deep).

super(side, side) — Square passes same value for both width & height.

isSquare() — Square inherits this from Rectangle (width===height always true for Square).

this.constructor.name — dynamic, returns actual class name even through inheritance.

Exercise 4 — Getters & Setters with Validation

Concept: [Getters / Setters](#) **Task:** Student class mein getters/setters add karo — marks validation aur computed grade ke saath:

```
class Student {
  #marks = [];
  #name;

  constructor(name) {
    this.name = name; // uses setter
  }

  // Setter with validation
  set name(val) {
    if (typeof val !== 'string' || val.trim().length < 2)
      throw new TypeError('Name must be 2+ chars');
    this.#name = val.trim();
  }

  get name() { return this.#name; }

  addMark(mark) {
    if (mark < 0 || mark > 100) throw new RangeError('Mark: 0-100');
    this.#marks.push(mark);
    return this;
  }

  get average() {
    if (!this.#marks.length) return 0;
    return +(this.#marks.reduce((a,b)=>a+b,0)/this.#marks.length).toFixed(1);
  }

  get grade() {
    const a = this.average;
    return a>=90?'A':a>=80?'A':a>=70?'B':a>=60?'C':'F';
  }

  get passed() { return this.average >= 40; }
  get report() {
    return `${this.name} | Avg: ${this.average} | Grade: ${this.grade} | ` +
      `${this.passed ? 'PASSED' : 'FAILED'}`;
  }
}

const s = new Student('Priya');
s.addMark(85).addMark(92).addMark(78).addMark(88);
s.average; // 85.8
s.grade; // 'A'
s.report; // 'Priya | Avg: 85.8 | Grade: A | PASSED'
// s.name = 'R'; // TypeError: Name must be 2+ chars
// s.addMark(150); // RangeError: Mark: 0-100
```

Key Insight / Answer

Getter without () — `s.average` not `s.average()` — feels like a property!

setter validates input — invalid data throw karo, silently ignore mat karo.

Method chaining: addMark returns 'this' — s.addMark(85).addMark(92) works.

#marks is private — s.#marks direct access nahi hota outside class.

Exercise 5 ★ — Static Factory Methods

Concept: [Static Methods](#) **Task:** Color class banao with static factory methods — different formats se create karna:

```
class Color {
  constructor(r, g, b) {
    this.r = Math.max(0, Math.min(255, r));
    this.g = Math.max(0, Math.min(255, g));
    this.b = Math.max(0, Math.min(255, b));
  }
  // Static factory methods
  static fromHex(hex) {
    const n = parseInt(hex.replace('#', ''), 16);
    return new Color((n>>16)&255, (n>>8)&255, n&255);
  }
  static fromCSS(css) {
    const [r,g,b] = css.match(/\d+/g).map(Number);
    return new Color(r, g, b);
  }
  static random() {
    return new Color(
      Math.floor(Math.random()*256),
      Math.floor(Math.random()*256),
      Math.floor(Math.random()*256)
    );
  }
  static mix(c1, c2) {
    return new Color(
      Math.round((c1.r+c2.r)/2),
      Math.round((c1.g+c2.g)/2),
      Math.round((c1.b+c2.b)/2)
    );
  }
  // Getters
  get hex() {
    return '#' + [this.r,this.g,this.b]
      .map(v=>v.toString(16).padStart(2, '0')).join('');
  }
  get rgb() { return `rgb(${this.r},${this.g},${this.b})`; }
  get isDark(){ return (this.r*299+this.g*587+this.b*114)/1000 < 128; }
}

const red = Color.fromHex('#ff0000');
const blue = Color.fromCSS('rgb(0,0,255)');
const mixed = Color.mix(red, blue);
red.rgb; // 'rgb(255,0,0)'
blue.hex; // '#0000ff'
mixed.hex; // '#7f007f' (purple!)
red.isDark; // false
```

Key Insight / Answer

Static factory methods allow multiple ways to create an object — clean API.

constructor mein validation (Math.clamp pattern) — invalid RGB values allowed nahi.

isDark getter — luminance formula se determine karo text color (white ya black).

Exercise 6 — Mixin Compose Challenge

Concept: Mixins **Task:** EventEmitter aur Logger mixins banao aur Product class mein compose karo:

```
// EventEmitter mixin
const EventEmitter = (Base) => class extends Base {
  #handlers = {};
  on(event, fn) {
    this.#handlers[event] = this.#handlers[event] || [];
    this.#handlers[event].push(fn);
    return this;
  }
  emit(event, data) {
    (this.#handlers[event] || []).forEach(fn => fn(data));
    return this;
  }
};

// Logger mixin
const Logger = (Base) => class extends Base {
  #logs = [];
  log(msg) { this.#logs.push(`${new Date().toISOString()} ${msg}`); return this; }
  getLogs() { return [...this.#logs]; }
  clearLogs() { this.#logs = []; return this; }
};

// Base class
class Product {
  constructor(name, price, stock) {
    this.name = name;
    this.price = price;
    this.stock = stock;
  }
  sell(qty) {
    if (qty > this.stock) throw new Error('Out of stock');
    this.stock -= qty;
    return this;
  }
}

// Compose Product with EventEmitter + Logger
class SmartProduct extends EventEmitter(Logger(Product)) {}

const laptop = new SmartProduct('Laptop', 55000, 10);
laptop
  .on('sell', ({ qty, remaining }) =>
    laptop.log(`Sold ${qty} units. Stock: ${remaining}`))
  .on('sell', ({ qty }) =>
    console.log(`Sale alert: ${qty} units sold!`));

// Extend sell to emit event
const origSell = laptop.sell.bind(laptop);
laptop.sell = function(qty) {
  origSell(qty);
```

```
this.emit('sell', { qty, remaining: this.stock });  
return this;  
};  
laptop.sell(3).sell(2);  
laptop.getLogs(); // 2 log entries  
laptop.stock; // 5
```

Key Insight / Answer

SmartProduct = EventEmitter + Logger + Product — 3 behaviors, 1 class.

No deep inheritance — each mixin adds independent behavior.

Private #handlers per mixin — no naming conflicts between mixins.

This pattern is used in Node.js EventEmitter, Vue reactivity, etc.